WIKIPEDIA 25
25 years of the free encyclopedia

Recurrent neural network

In artificial neural networks, **recurrent neural networks** (**RNNs**) are designed for processing sequential data, such as text, speech, and time series,[1] where the order of elements is important. Unlike feedforward neural networks, which process inputs independently, RNNs utilize recurrent connections, where the output of a neuron at one time step is fed back as input to the network at the next time step. This enables RNNs to capture temporal dependencies and patterns within sequences.

The fundamental building block of RNN is the *recurrent unit*, which maintains a *hidden state*—a form of memory that is updated at each time step based on the current input and the previous hidden state. This feedback mechanism allows the network to learn from past inputs and incorporate that knowledge into its current processing. RNNs have been successfully applied to tasks such as unsegmented, connected handwriting recognition,[2] speech recognition,[3][4] natural language processing, and neural machine translation.[5][6]

However, traditional RNNs suffer from the vanishing gradient problem, which limits their ability to learn long-range dependencies. This issue was addressed by the development of the long short-term memory (LSTM) architecture in 1997, making it the standard RNN variant for handling long-term dependencies. Later, gated recurrent units (GRUs) were introduced as a more computationally efficient alternative.

In recent years, transformers, which rely on self-attention mechanisms instead of recurrence, have become the dominant architecture for many sequence-processing tasks, particularly in natural language processing, due to their superior handling of long-range dependencies and greater parallelizability. Nevertheless, RNNs remain relevant for applications where computational efficiency, real-time processing, or the inherent sequential nature of data is crucial.

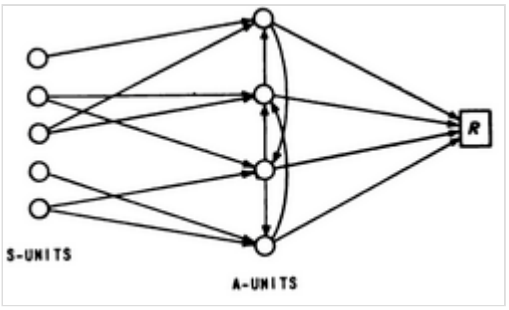
History

Before modern

One origin of RNN was neuroscience. The word "recurrent" is used to describe loop-like structures in anatomy. In 1901, Cajal observed "recurrent semicircles" in the cerebellar cortex formed by parallel fiber, Purkinje cells, and granule cells.[7][8] In 1933, Lorente de Nó discovered "recurrent, reciprocal connections" by Golgi's method, and proposed that excitatory loops explain certain aspects of the vestibulo-ocular reflex.[9][10] During 1940s, multiple people proposed the existence of feedback in the brain, which was a contrast to the previous understanding of the neural system as a purely feedforward structure. Hebb considered "reverberating circuit" as an explanation for short-term memory.[11] The McCulloch and Pitts paper (1943), which proposed the McCulloch-Pitts neuron model, considered networks that contains cycles. The current activity of such networks can be affected by activity indefinitely far in the past.[12] They were both interested in closed loops as possible explanations for e.g.

epilepsy and causalgia.[13][14] Recurrent inhibition was proposed in 1946 as a negative feedback mechanism in motor control. Neural feedback loops were a common topic of discussion at the Macy conferences.[15] See [16] for an extensive review of recurrent neural network models in neuroscience.

Frank Rosenblatt in 1960 published "close-loop cross-coupled perceptrons", which are 3-layered perceptron networks whose middle layer contains recurrent connections that change by a Hebbian learning rule.[18]:73–75 Later, in *Principles of Neurodynamics* (1961), he described "closed-loop cross-coupled" and "back-coupled" perceptron networks, and made theoretical and experimental studies for Hebbian learning in these networks,[17]:Chapter 19, 21 and noted that a fully cross-coupled perceptron network is equivalent to an infinitely deep feedforward network.[17]:Section 19.11



A close-loop cross-coupled perceptron network[17]:403,Fig. 47

Similar networks were published by Kaoru Nakano in 1971,[19][20] Shun'ichi Amari in 1972,[21] and William A. Little in 1974,[22] who was acknowledged by Hopfield in his 1982 paper.

Another origin of RNN was statistical mechanics. The Ising model was developed by Wilhelm Lenz[23] and Ernst Ising[24] in the 1920s[25] as a simple statistical mechanical model of magnets at equilibrium. Glauber in 1963 studied the Ising model evolving in time, as a process towards equilibrium (Glauber dynamics), adding in the component of time.[26]

The Sherrington–Kirkpatrick model of spin glass, published in 1975,[27] is the Hopfield network with random initialization. Sherrington and Kirkpatrick found that it is highly likely for the energy function of the SK model to have many local minima. In the 1982 paper, Hopfield applied this recently developed theory to study the Hopfield network with binary activation functions.[28] In a 1984 paper he extended this to continuous activation functions.[29] It became a standard model for the study of neural networks through statistical mechanics.[30][31]

Modern

During the resurgence of neural networks in the 1980s, recurrent networks were studied again. Two influential architectures from this period were the Jordan network, proposed by Michael I. Jordan in 1986, and the Elman network, introduced by Jeffrey Elman in 1990.[32][33]

Long short-term memory (LSTM) was introduced by Sepp Hochreiter and Jürgen Schmidhuber in 1997 to address difficulties in learning long-range dependencies with recurrent networks.[34] The same year, Mike Schuster and Kuldip K. Paliwal introduced bidirectional recurrent neural networks (BRNNs), which process a sequence in both forward and backward directions.[35]

Bidirectionality was subsequently combined with LSTM, and by the mid-2000s bidirectional LSTM networks had achieved strong results in speech recognition.[36] They improved large-vocabulary speech recognition[3][4] and text-to-speech synthesis[37] and were used in Google voice search.[38]

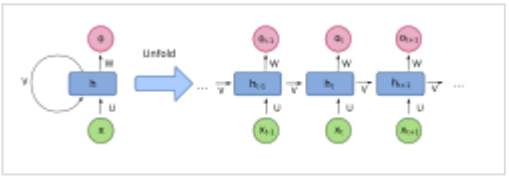
Recurrent networks also proved effective for language modeling. In 2010, Tomáš Mikolov and co-authors demonstrated that recurrent neural-network language models could substantially outperform conventional n-gram models.[39]

In 2014, Kyunghyun Cho and co-authors proposed an RNN encoder–decoder for machine translation,[40] while another 2014 study demonstrated sequence-to-sequence learning using LSTMs.[41] They became state of the art in machine translation, and were instrumental in the development of attention mechanisms and transformers.

Configurations

An RNN-based model can be factored into two parts: configuration and architecture. Multiple RNNs can be combined in a data flow, and the data flow itself is the configuration. Each RNN itself may have any architecture, including LSTM, GRU, etc.

Standard



Compressed (left) and unfolded (right) basic recurrent neural network

RNNs come in many variants. Abstractly speaking, an RNN is a function f_θ of type $(x_t, h_t) \mapsto (y_t, h_{t+1})$, where

- x_t: input vector;
- h_t: hidden vector;
- y_t: output vector;
- θ: neural network parameters.

In words, it is a neural network that maps an input x_t into an output y_t, with the hidden vector h_t playing the role of "memory", a partial record of all previous input-output pairs. At each step, it transforms input to an output, and modifies its "memory" to help it to better perform future processing.

The illustration to the right may be misleading to many because practical neural network topologies are frequently organized in "layers" and the drawing gives that appearance. However, what appears to be layers are, in fact, different steps in time, "unfolded" to produce the appearance of layers.

Stacked RNN



Stacked RNN

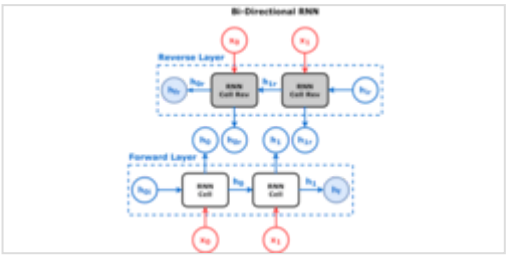
A **stacked RNN**, or **deep RNN**, is composed of multiple RNNs stacked one above the other. Abstractly, it is structured as follows

1. Layer 1 has hidden vector $h_{1,t}$, parameters θ_1, and maps $f_{\theta_1} : (x_{0,t}, h_{1,t}) \mapsto (x_{1,t}, h_{1,t+1})$.
2. Layer 2 has hidden vector $h_{2,t}$, parameters θ_2, and maps $f_{\theta_2} : (x_{1,t}, h_{2,t}) \mapsto (x_{2,t}, h_{2,t+1})$.
3. ...
4. Layer n has hidden vector $h_{n,t}$, parameters θ_n, and maps $f_{\theta_n} : (x_{n-1,t}, h_{n,t}) \mapsto (x_{n,t}, h_{n,t+1})$.

Each layer operates as a stand-alone RNN, and each layer's output sequence is used as the input sequence to the layer above. There is no conceptual limit to the depth of stacked RNN.

Bidirectional

A **bidirectional RNN** (biRNN) is composed of two RNNs, one processing the input sequence in one direction, and another in the opposite direction. Abstractly, it is structured as follows:



Bidirectional RNN

- The forward RNN processes in one direction:

$$f_\theta(x_0, h_0) = (y_0, h_1), f_\theta(x_1, h_1) = (y_1, h_2), \dots$$

- The backward RNN processes in the opposite direction:

$$f'_{\theta'}(x_N, h'_N) = (y'_N, h'_{N-1}), f'_{\theta'}(x_{N-1}, h'_{N-1}) = (y'_{N-1}, h'_{N-2}), \dots$$

The two output sequences are then concatenated to give the total output: $((y_0, y'_0), (y_1, y'_1), \dots, (y_N, y'_N))$.

Bidirectional RNN allows the model to process a token both in the context of what came before it and what came after it. By stacking multiple bidirectional RNNs together, the model can process a token increasingly contextually. The ELMo model (2018)[42] is a stacked bidirectional LSTM which takes character-level as inputs and produces word-level embeddings.

Encoder-decoder

Two RNNs can be run front-to-back in an **encoder-decoder** configuration. The encoder RNN processes an input sequence into a sequence of hidden vectors, and the decoder RNN processes the sequence of hidden vectors to an output sequence, with an optional attention mechanism. This was used to construct state of the art neural machine translators during the 2014–2017 period. This was an instrumental step towards the development of transformers.[43]



A decoder without an encoder



Encoder-decoder RNN without attention mechanism

PixelRNN

An RNN may process data with more than one dimension. PixelRNN processes two-dimensional data, with many possible directions.[44] For example, the row-by-row direction processes an $n \times n$ grid of vectors $x_{i,j}$ in the following order:

$$x_{1,1}, x_{1,2}, \dots, x_{1,n}, x_{2,1}, x_{2,2}, \dots, x_{2,n}, \dots, x_{n,n}$$

The **diagonal BiLSTM** uses two LSTMs to process the same grid. One processes it from the top-left corner to the bottom-right, such that it processes $x_{i,j}$ depending on its hidden state and cell state on the top and the left side: $h_{i-1,j}, c_{i-1,j}$ and $h_{i,j-1}, c_{i,j-1}$. The other processes it from the top-right corner to the bottom-left.

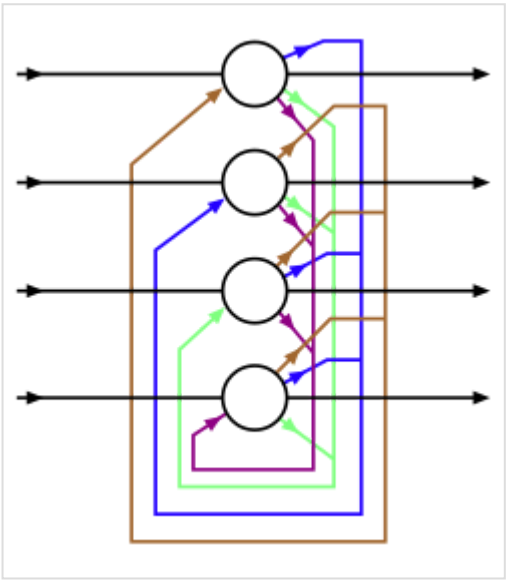


Encoder-decoder RNN with attention mechanism

Architectures

Fully recurrent

Fully recurrent neural networks (FRNN) connect the outputs of all neurons to the inputs of all neurons. In other words, it is a fully connected network. This is the most general neural network topology, because all other topologies can be represented by setting some connection weights to zero to simulate the lack of connections between those neurons.

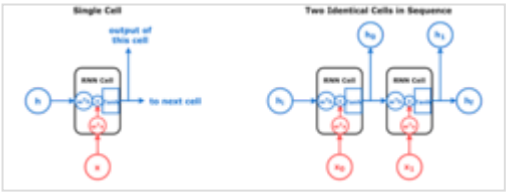


A fully connected RNN with 4 neurons

Hopfield

The **Hopfield network** is an RNN in which all connections across layers are equally sized. It requires stationary inputs and is thus not a general RNN, as it does not process sequences of patterns. However, it guarantees that it will converge. If the connections are trained using Hebbian learning, then the Hopfield network can perform as robust content-addressable memory, resistant to connection alteration.

Elman networks and Jordan networks

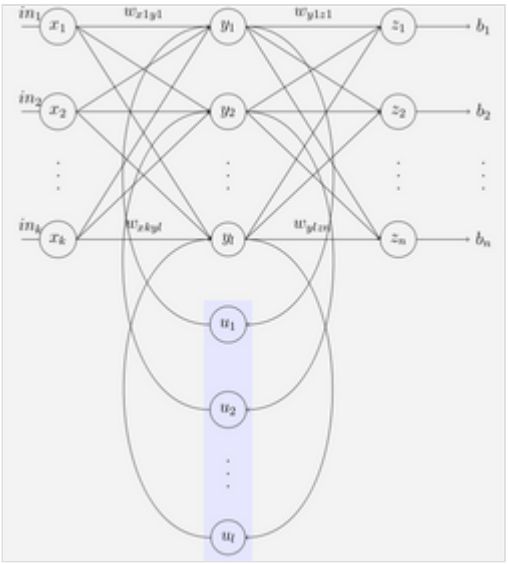


A simple Elman network where $\sigma_h = \tanh, \sigma_y = \text{Identity}$

An **Elman network** is a three-layer network (arranged horizontally as *x*, *y*, and *z* in the illustration) with the addition of a set of context units (*u* in the illustration). The middle (hidden) layer is connected to these context units fixed with a weight of one.[45] At each time step, the input is fed forward and a learning rule is applied. The fixed back-connections save a copy of the previous values of the hidden units in the context units (since they propagate over the connections before the learning rule is applied). Thus the network can maintain a sort of state, allowing it to perform tasks such as sequence-prediction that are beyond the power of a standard multilayer perceptron.

Jordan networks are similar to Elman networks. The context units are fed from the output layer instead of the hidden layer. The context units in a Jordan network are also called the state layer. They have a recurrent connection to themselves.[45]

Elman and Jordan networks are also known as "Simple recurrent networks" (SRN).



The Elman network

Elman network[46]

$$h_t = \sigma_h(W_h x_t + U_h h_{t-1} + b_h)$$
$$y_t = \sigma_y(W_y h_t + b_y)$$

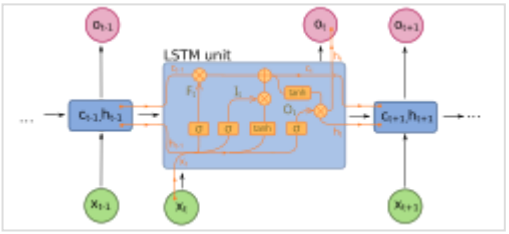
Jordan network[47]

$$h_t = \sigma_h(W_h x_t + U_h s_t + b_h)$$
$$y_t = \sigma_y(W_y h_t + b_y)$$
$$s_t = \sigma_s(W_{s,s} s_{t-1} + W_{s,y} y_{t-1} + b_s)$$

Variables and functions

- x_t: input vector
- h_t: hidden layer vector
- s_t: "state" vector,
- y_t: output vector
- W, U and b: parameter matrices and vector
- σ: Activation functions

Long short-term memory



Long short-term memory unit

Long short-term memory (LSTM) is the most widely used RNN architecture. It was designed to solve the vanishing gradient problem. LSTM is normally augmented by recurrent gates called "forget gates".[48] LSTM prevents backpropagated errors from vanishing or exploding.[49] Instead, errors can flow backward through unlimited numbers of virtual layers unfolded in space. That is, LSTM can learn tasks that require memories of events that happened thousands or even millions of discrete time steps earlier. Problem-specific LSTM-like topologies can be evolved.[50] LSTM works even given long delays between significant events and can handle signals that mix low and high-frequency components.

Many applications use stacks of LSTMs,[51] for which it is called "deep LSTM". LSTM can learn to recognize context-sensitive languages unlike previous models based on hidden Markov models (HMM) and similar concepts.[52]

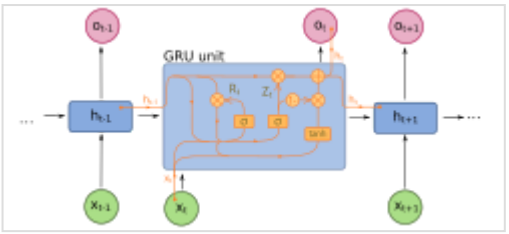
Gated recurrent unit

Gated recurrent unit (GRU), introduced in 2014, was designed as a simplification of LSTM. They are used in the full form and several further simplified variants.[53][54] They have fewer parameters than LSTM, as they lack an output gate.[55]

Their performance on polyphonic music modeling and speech signal modeling was found to be similar to that of long short-term memory.[56] There does not appear to be particular performance difference between LSTM and GRU.[56][57]

Bidirectional associative memory

Introduced by Bart Kosko,[58] a bidirectional associative memory (BAM) network is a variant of a Hopfield network that stores associative data as a vector. The bidirectionality comes from passing information through a matrix and its transpose. Typically, bipolar encoding is preferred to binary encoding of the associative pairs. Recently, stochastic BAM models using Markov stepping are optimized for increased network stability and relevance to real-world applications.[59]



Gated recurrent unit

A BAM network has two layers, either of which can be driven as an input to recall an association and produce an output on the other layer.[60]

Echo state

Echo state networks (ESN) have a sparsely connected random hidden layer. The weights of output neurons are the only part of the network that can change (be trained). ESNs are good at reproducing certain time series.[61] A variant for spiking neurons is known as a liquid state machine.[62]

Recursive

A **recursive neural network**[63] is created by applying the same set of weights recursively over a differentiable graph-like structure by traversing the structure in topological order. Such networks are typically also trained by the reverse mode of automatic differentiation.[64][65] They can process distributed representations of structure, such as logical terms. A special case of recursive neural networks is the RNN whose structure corresponds to a linear chain. Recursive neural networks have been applied to natural language processing.[66] The *recursive neural tensor network* uses a tensor-based composition function for all nodes in the tree.[67]

Neural Turing machines

Neural Turing machines (NTMs) are a method of extending recurrent neural networks by coupling them to external memory resources with which they interact. The combined system is analogous to a Turing machine or Von Neumann architecture but is differentiable end-to-end, allowing it to be efficiently trained with gradient descent.[68]

Differentiable neural computers (DNCs) are an extension of neural Turing machines, allowing for the usage of fuzzy amounts of each memory address and a record of chronology.[69]

Neural network pushdown automata (NNPDA) are similar to NTMs, but tapes are replaced by analog stacks that are differentiable and trained. In this way, they are similar in complexity to recognizers of context free grammars (CFGs).[70]

Recurrent neural networks are Turing complete and can run arbitrary programs to process arbitrary sequences of inputs.[71]

Training

Teacher forcing

An RNN can be trained into a conditionally generative model of sequences, aka **autoregression**.

Concretely, let us consider the problem of machine translation, that is, given a sequence $(x_1, x_2, \ldots, x_n)$ of English words, the model is to produce a sequence $(y_1, \ldots, y_m)$ of French words. It is to be solved by a seq2seq model.



Encoder-decoder RNN without attention mechanism. Teacher forcing is shown in red.

Now, during training, the encoder half of the model would first ingest $(x_1, x_2, \ldots, x_n)$, then the decoder half would start generating a sequence $(\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_l)$. The problem is that if the model makes a mistake early on, say at $\hat{y}_2$, then subsequent tokens are likely to also be mistakes. This makes it inefficient for the model to obtain a learning signal, since the model would mostly learn to shift $\hat{y}_2$ towards y_2, but not the others.

Teacher forcing makes it so that the decoder uses the correct output sequence for generating the next entry in the sequence. So for example, it would see $(y_1, \ldots, y_k)$ in order to generate $\hat{y}_{k+1}$.

Gradient descent

Gradient descent is a first-order iterative optimization algorithm for finding the minimum of a function. In neural networks, it can be used to minimize the error term by changing each weight in proportion to the derivative of the error with respect to that weight, provided the non-linear activation functions are differentiable.

The standard method for training RNN by gradient descent is the "backpropagation through time" (BPTT) algorithm, which is a special case of the general algorithm of backpropagation. A more computationally expensive online variant is called "Real-Time Recurrent Learning" or RTRL,[72][73] which is an instance of automatic differentiation in the forward accumulation mode with stacked tangent vectors. Unlike BPTT, this algorithm is local in time but not local in space.

In this context, local in space means that a unit's weight vector can be updated using only information stored in the connected units and the unit itself such that update complexity of a single unit is linear in the dimensionality of the weight vector. Local in time means that the updates take place continually (on-line) and depend only on the most recent time step rather than on multiple time steps within a given time horizon as in BPTT. Biological neural networks appear to be local with respect to both time and space.[74][75]

For recursively computing the partial derivatives, RTRL has a time-complexity of O(number of hidden x number of weights) per time step for computing the Jacobian matrices, while BPTT only takes O(number of weights) per time step, at the cost of storing all forward activations within the given time horizon.[76]

An online hybrid between BPTT and RTRL with intermediate complexity exists,[77][78] along with variants for continuous time.[79]

A major problem with gradient descent for standard RNN architectures is that error gradients vanish exponentially quickly with the size of the time lag between important events.[49][80] LSTM combined with a BPTT/RTRL hybrid learning method attempts to overcome these problems.[81] This problem is also solved in the independently recurrent neural network (IndRNN)[82] by reducing the context of a neuron to its own past state and the cross-neuron information can then be explored in the following layers. Memories of different ranges including long-term memory can be learned without the gradient vanishing and exploding problems.

The online algorithm called **causal recursive backpropagation** (CRBP), implements and combines BPTT and RTRL paradigms for locally recurrent networks.[83] It works with the most general locally recurrent networks. The CRBP algorithm can minimize the global error term. This fact improves the stability of the algorithm, providing a unifying view of gradient calculation techniques for recurrent networks with local feedback.

One approach to gradient information computation in RNNs with arbitrary architectures is based on signal-flow graphs diagrammatic derivation.[84] It uses the BPTT batch algorithm, based on Lee's theorem for network sensitivity calculations.[85] It was proposed by Wan and Beaufays, while its fast online version was proposed by Campolucci, Uncini and Piazza.[85]

Connectionist temporal classification

The connectionist temporal classification (CTC)[86] is a specialized loss function for training RNNs for sequence modeling problems where the timing is variable.[87]

Global optimization methods

Training the weights in a neural network can be modeled as a non-linear global optimization problem. A target function can be formed to evaluate the fitness or error of a particular weight vector as follows: First, the weights in the network are set according to the weight vector. Next, the network is evaluated against the training sequence. Typically, the sum-squared difference between the predictions and the target values specified in the training sequence is used to represent the error of the current weight vector. Arbitrary global optimization techniques may then be used to minimize this target function.

The most common global optimization method for training RNNs is genetic algorithms, especially in unstructured networks.[88][89][90]

Initially, the genetic algorithm is encoded with the neural network weights in a predefined manner where one gene in the chromosome represents one weight link. The whole network is represented as a single chromosome. The fitness function is evaluated as follows:

- Each weight encoded in the chromosome is assigned to the respective weight link of the network.
- The training set is presented to the network which propagates the input signals forward.
- The mean-squared error is returned to the fitness function.
- This function drives the genetic selection process.

Many chromosomes make up the population; therefore, many different neural networks are evolved until a stopping criterion is satisfied. A common stopping scheme can be:

- When the neural network has learned a certain percentage of the training data.
- When the minimum value of the mean-squared-error is satisfied.
- When the maximum number of training generations has been reached.

The fitness function evaluates the stopping criterion as it receives the mean-squared error reciprocal from each network during training. Therefore, the goal of the genetic algorithm is to maximize the fitness function, reducing the mean-squared error.

Other global (and/or evolutionary) optimization techniques may be used to seek a good set of weights, such as simulated annealing or particle swarm optimization.

Other architectures

Independently RNN (IndRNN)

The independently recurrent neural network (IndRNN)[82] addresses the gradient vanishing and exploding problems in the traditional fully connected RNN. Each neuron in one layer only receives its own past state as context information (instead of full connectivity to all other neurons in this layer) and thus neurons are independent of each other's history. The gradient backpropagation can be regulated to avoid gradient vanishing and exploding in order to keep long or short-term memory. The cross-neuron information is explored in the next layers. IndRNN can be robustly trained with non-saturated nonlinear functions such as ReLU. Deep networks can be trained using skip connections.

Neural history compressor

The *neural history compressor* is an unsupervised stack of RNNs.[91] At the input level, it learns to predict its next input from the previous inputs. Only unpredictable inputs of some RNN in the hierarchy become inputs to the next higher level RNN, which therefore recomputes its internal state only rarely. Each higher level RNN thus studies a compressed representation of the information in the RNN below. This is done such that the input sequence can be precisely reconstructed from the representation at the highest level.

The system effectively minimizes the description length or the negative logarithm of the probability of the data.[92] Given a lot of learnable predictability in the incoming data sequence, the highest level RNN can use supervised learning to easily classify even deep sequences with long intervals between important events.

It is possible to distill the RNN hierarchy into two RNNs: the "conscious" chunker (higher level) and the "subconscious" automatizer (lower level).[91] Once the chunker has learned to predict and compress inputs that are unpredictable by the automatizer, then the automatizer can be forced in the next learning phase to predict or imitate through additional units the hidden units of the more slowly changing chunker. This makes it easy for the automatizer to learn appropriate, rarely changing memories across long intervals. In turn, this helps the automatizer to make many of its once unpredictable inputs predictable, such that the chunker can focus on the remaining unpredictable events.[91]

A generative model partially overcame the vanishing gradient problem[49] of automatic differentiation or backpropagation in neural networks in 1992. In 1993, such a system solved a "Very Deep Learning" task that required more than 1000 subsequent layers in an RNN unfolded in time.[93]

Second order RNNs

Second-order RNNs use higher order weights w_{ijk} instead of the standard w_{ij} weights, and states can be a product. This allows a direct mapping to a finite-state machine both in training, and representation.[94][95] Long short-term memory is an example of this but has no such formal mappings or proof of stability.

Hierarchical recurrent neural network

Hierarchical recurrent neural networks (HRNN) connect their neurons in various ways to decompose hierarchical behavior into useful subprograms.[91][96] Such hierarchical structures of cognition are present in theories of memory presented by philosopher Henri Bergson, whose philosophical views have inspired hierarchical models.[97]

Hierarchical recurrent neural networks are useful in forecasting, helping to predict disaggregated inflation components of the consumer price index (CPI). The HRNN model leverages information from higher levels in the CPI hierarchy to enhance lower-level predictions. Evaluation of a substantial dataset from the US CPI-U index demonstrates the superior performance of the HRNN model compared to various established inflation prediction methods.[98]

Recurrent multilayer perceptron network

Generally, a recurrent multilayer perceptron network (RMLPN) consists of cascaded subnetworks, each containing multiple layers of nodes. Each subnetwork is feed-forward except for the last layer, which can have feedback connections. Each of these subnets is connected only by feed-forward connections.[99]

Multiple timescales model

A multiple timescales recurrent neural network (MTRNN) is a neural-based computational model that can simulate the functional hierarchy of the brain through self-organization depending on the spatial connection between neurons and on distinct types of neuron activities, each with distinct time properties.[100][101] With such varied neuronal activities, continuous sequences of any set of behaviors are segmented into reusable primitives, which in turn are flexibly integrated into diverse sequential behaviors. The biological approval of such a type of hierarchy was discussed in the memory-prediction theory of brain function by Hawkins in his book *On Intelligence*. Such a hierarchy also agrees with theories of memory posited by philosopher Henri Bergson, which have been incorporated into an MTRNN model.[97][102]

Memristive networks

Greg Snider of HP Labs describes a system of cortical computing with memristive nanodevices.[103] The memristors (memory resistors) are implemented by thin film materials in which the resistance is electrically tuned via the transport of ions or oxygen vacancies within the film. DARPA's SyNAPSE

project has funded IBM Research and HP Labs, in collaboration with the Boston University Department of Cognitive and Neural Systems (CNS), to develop neuromorphic architectures that may be based on memristive systems. Memristive networks are a particular type of physical neural network that have very similar properties to (Little-)Hopfield networks, as they have continuous dynamics, a limited memory capacity and natural relaxation via the minimization of a function which is asymptotic to the Ising model. In this sense, the dynamics of a memristive circuit have the advantage compared to a Resistor-Capacitor network to have a more interesting non-linear behavior. From this point of view, engineering analog memristive networks account for a peculiar type of neuromorphic engineering in which the device behavior depends on the circuit wiring or topology. The evolution of these networks can be studied analytically using variations of the Caravelli-Traversa-Di Ventra equation.[104]

Continuous-time

A continuous-time recurrent neural network (CTRNN) uses a system of ordinary differential equations to model the effects on a neuron of the incoming inputs. They are typically analyzed by dynamical systems theory. Many RNN models in neuroscience are continuous-time.[16]

For a neuron i in the network with activation y_i, the rate of change of activation is given by:

$$\tau_i \dot{y}_i = -y_i + \sum_{j=1}^{n} w_{ji}\sigma(y_j - \Theta_j) + I_i(t)$$

Where:

- τ_i : Time constant of postsynaptic node
- y_i : Activation of postsynaptic node
- $\dot{y}_i$: Rate of change of activation of postsynaptic node
- w_{ji} : Weight of connection from pre to postsynaptic node
- $\sigma(x)$: Sigmoid of x e.g. $\sigma(x) = 1/(1 + e^{-x})$.
- y_j : Activation of presynaptic node
- Θ_j : Bias of presynaptic node
- $I_i(t)$: Input (if any) to node

CTRNNs have been applied to evolutionary robotics where they have been used to address vision,[105] co-operation,[106] and minimal cognitive behaviour.[107]

Note that, by the Shannon sampling theorem, discrete-time recurrent neural networks can be viewed as continuous-time recurrent neural networks where the differential equations have transformed into equivalent difference equations.[108] This transformation can be thought of as occurring after the post-synaptic node activation functions $y_i(t)$ have been low-pass filtered but prior to sampling.

They are in fact recursive neural networks with a particular structure: that of a linear chain. Whereas recursive neural networks operate on any hierarchical structure, combining child representations into parent representations, recurrent neural networks operate on the linear progression of time, combining the previous time step and a hidden representation into the representation for the current time step.

From a time-series perspective, RNNs can appear as nonlinear versions of finite impulse response and infinite impulse response filters and also as a nonlinear autoregressive exogenous model (NARX).[109] RNN has infinite impulse response whereas convolutional neural network has finite impulse response. Both classes of networks exhibit temporal dynamic behavior.[110] A finite impulse recurrent network is a directed acyclic graph that can be unrolled and replaced with a strictly feedforward neural network, while an infinite impulse recurrent network is a directed cyclic graph that cannot be unrolled.

The effect of memory-based learning for the recognition of sequences can also be implemented by a more biological-based model which uses the silencing mechanism exhibited in neurons with a relatively high frequency spiking activity.[111]

Additional stored states and the storage under direct control by the network can be added to both infinite-impulse and finite-impulse networks. Another network or graph can also replace the storage if that incorporates time delays or has feedback loops. Such controlled states are referred to as gated states or gated memory and are part of long short-term memory networks (LSTMs) and gated recurrent units. This is also called Feedback Neural Network (FNN).

Libraries

Modern libraries provide runtime-optimized implementations of the above functionality or allow to speed up the slow loop by just-in-time compilation.

- Apache Singa
- Caffe: Created by the Berkeley Vision and Learning Center (BVLC). It supports both CPU and GPU. Developed in C++, and has Python and MATLAB wrappers.
- Chainer: Fully in Python, production support for CPU, GPU, distributed training.
- Deeplearning4j: Deep learning in Java and Scala on multi-GPU-enabled Spark.
- Flux: includes interfaces for RNNs, including GRUs and LSTMs, written in Julia.
- Keras: High-level API, providing a wrapper to many other deep learning libraries.
- Microsoft Cognitive Toolkit
- MXNet: an open-source deep learning framework used to train and deploy deep neural networks.
- PyTorch: Tensors and Dynamic neural networks in Python with GPU acceleration.
- TensorFlow: Apache 2.0-licensed Theano-like library with support for CPU, GPU and Google's proprietary TPU,[112] mobile
- Theano: A deep-learning library for Python with an API largely compatible with the NumPy library.
- Torch: A scientific computing framework with support for machine learning algorithms, written in C and Lua.

Applications

Applications of recurrent neural networks include:

- Machine translation[113]
- Robot control[114]

- Time series prediction[115][116][117]
- Speech recognition[118][119][120]
- Speech synthesis[121]
- Brain–computer interfaces[122]
- Time series anomaly detection[123]
- Text-to-Video model[124]
- Energy forecasting[125]
- Rhythm learning[126]
- Music composition[127]
- Grammar learning[128][52][129]
- Handwriting recognition[130][131]
- Human action recognition[132]
- Protein homology detection[133]
- Predicting subcellular localization of proteins[134]
- Several prediction tasks in the area of business process management[135]
- Prediction in medical care pathways[136]
- Predictions of fusion plasma disruptions in reactors (Fusion Recurrent Neural Network (FRNN) code) [137]

References

1. Tealab, Ahmed (2018-12-01). "Time series forecasting using artificial neural networks methodologies: A systematic review" (https://doi.org/10.1016%2Fj.fcij.2018.10.003). *Future Computing and Informatics Journal*. **3** (2): 334–340. doi:10.1016/j.fcij.2018.10.003 (https://doi.org/10.1016%2Fj.fcij.2018.10.003). ISSN 2314-7288 (https://search.worldcat.org/issn/2314-7288).
2. Graves, Alex; Liwicki, Marcus; Fernandez, Santiago; Bertolami, Roman; Bunke, Horst; Schmidhuber, Jürgen (2009). "A Novel Connectionist System for Improved Unconstrained Handwriting Recognition" (http://www.idsia.ch/~juergen/tpami_2008.pdf) (PDF). *IEEE Transactions on Pattern Analysis and Machine Intelligence*. **31** (5): 855–868. doi:10.1109/tpami.2008.137 (https://doi.org/10.1109%2Ftpami.2008.137). PMID 19299860 (https://pubmed.ncbi.nlm.nih.gov/19299860). S2CID 14635907 (https://api.semanticscholar.org/CorpusID:14635907).
3. Sak, Haşim; Senior, Andrew; Beaufays, Françoise (2014). "Long Short-Term Memory recurrent neural network architectures for large scale acoustic modeling" (https://research.google.com/pubs/archive/43905.pdf) (PDF). Google Research.
4. Li, Xiangang; Wu, Xihong (2014-10-15). "Constructing Long Short-Term Memory based Deep Recurrent Neural Networks for Large Vocabulary Speech Recognition". arXiv:1410.4281 (https://arxiv.org/abs/1410.4281) [cs.CL (https://arxiv.org/archive/cs.CL)].
5. Dupond, Samuel (2019). "A thorough review on the current advance of neural network structures" (https://www.sciencedirect.com/journal/annual-reviews-in-control). *Annual Reviews in Control*. **14**: 200–230.

6. Abiodun, Oludare Isaac; Jantan, Aman; Omolara, Abiodun Esther; Dada, Kemi Victoria; Mohamed, Nachaat Abdelatif; Arshad, Humaira (2018-11-01). "State-of-the-art in artificial neural network applications: A survey" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6260436). *Heliyon*. **4** (11) e00938. Bibcode:2018Heliy...400938A (https://ui.adsabs.harvard.edu/abs/2018Heliy...400938A). doi:10.1016/j.heliyon.2018.e00938 (https://doi.org/10.1016%2Fj.heliyon.2018.e00938). ISSN 2405-8440 (https://search.worldcat.org/issn/2405-8440). PMC 6260436 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6260436). PMID 30519653 (https://pubmed.ncbi.nlm.nih.gov/30519653).
7. Espinosa-Sanchez, Juan Manuel; Gomez-Marin, Alex; de Castro, Fernando (2023-07-05). "The Importance of Cajal's and Lorente de Nó's Neuroscience to the Birth of Cybernetics" (http://journals.sagepub.com/doi/10.1177/10738584231179932). *The Neuroscientist*. **31** (1): 14–30. doi:10.1177/10738584231179932 (https://doi.org/10.1177%2F10738584231179932). hdl:10261/348372 (https://hdl.handle.net/10261%2F348372). ISSN 1073-8584 (https://search.worldcat.org/issn/1073-8584). PMID 37403768 (https://pubmed.ncbi.nlm.nih.gov/37403768).
8. Ramón y Cajal, Santiago (1909). *Histologie du système nerveux de l'homme & des vertébrés* (https://archive.org/details/b2129592x_0002/page/n159/mode/2up). Vol. II. Foyle Special Collections Library King's College London. Paris : A. Maloine. p. 149.
9. de NÓ, R. Lorente (1933-08-01). "Vestibulo-Ocular Reflex Arc" (http://archneurpsyc.jamanetwork.com/article.aspx?doi=10.1001/archneurpsyc.1933.02240140009001). *Archives of Neurology and Psychiatry*. **30** (2): 245. doi:10.1001/archneurpsyc.1933.02240140009001 (https://doi.org/10.1001%2Farchneurpsyc.1933.02240140009001). ISSN 0096-6754 (https://search.worldcat.org/issn/0096-6754).
10. Larriva-Sahd, Jorge A. (2014-12-03). "Some predictions of Rafael Lorente de Nó 80 years later" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4253658). *Frontiers in Neuroanatomy*. **8**: 147. doi:10.3389/fnana.2014.00147 (https://doi.org/10.3389%2Ffnana.2014.00147). ISSN 1662-5129 (https://search.worldcat.org/issn/1662-5129). PMC 4253658 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4253658). PMID 25520630 (https://pubmed.ncbi.nlm.nih.gov/25520630).
11. "reverberating circuit" (https://www.oxfordreference.com/display/10.1093/oi/authority.20110803100417461). *Oxford Reference*. Retrieved 2024-07-27.
12. McCulloch, Warren S.; Pitts, Walter (December 1943). "A logical calculus of the ideas immanent in nervous activity" (http://link.springer.com/10.1007/BF02478259). *The Bulletin of Mathematical Biophysics*. **5** (4): 115–133. doi:10.1007/BF02478259 (https://doi.org/10.1007%2FBF02478259). ISSN 0007-4985 (https://search.worldcat.org/issn/0007-4985).
13. Moreno-Díaz, Roberto; Moreno-Díaz, Arminda (April 2007). "On the legacy of W.S. McCulloch" (https://linkinghub.elsevier.com/retrieve/pii/S0303264706002152). *Biosystems*. **88** (3): 185–190. Bibcode:2007BiSys..88..185M (https://ui.adsabs.harvard.edu/abs/2007BiSys..88..185M). doi:10.1016/j.biosystems.2006.08.010 (https://doi.org/10.1016%2Fj.biosystems.2006.08.010). PMID 17184902 (https://pubmed.ncbi.nlm.nih.gov/17184902).
14. Arbib, Michael A (December 2000). "Warren McCulloch's Search for the Logic of the Nervous System" (https://muse.jhu.edu/article/46496). *Perspectives in Biology and Medicine*. **43** (2): 193–216. doi:10.1353/pbm.2000.0001 (https://doi.org/10.1353%2Fpbm.2000.0001). ISSN 1529-8795 (https://search.worldcat.org/issn/1529-8795). PMID 10804585 (https://pubmed.ncbi.nlm.nih.gov/10804585).
15. Renshaw, Birdsey (1946-05-01). "Central Effects of Centripetal Impulses in Axons of Spinal Ventral Roots" (https://www.physiology.org/doi/10.1152/jn.1946.9.3.191). *Journal of Neurophysiology*. **9** (3): 191–204. doi:10.1152/jn.1946.9.3.191 (https://doi.org/10.1152%2Fjn.1946.9.3.191). ISSN 0022-3077 (https://search.worldcat.org/issn/0022-3077). PMID 21028162 (https://pubmed.ncbi.nlm.nih.gov/21028162).

16. Grossberg, Stephen (2013-02-22). "Recurrent Neural Networks" (https://doi.org/10.4249%2Fscholarpedia.1888). *Scholarpedia*. **8** (2): 1888. Bibcode:2013SchpJ...8.1888G (https://ui.adsabs.harvard.edu/abs/2013SchpJ...8.1888G). doi:10.4249/scholarpedia.1888 (https://doi.org/10.4249%2Fscholarpedia.1888). ISSN 1941-6016 (https://search.worldcat.org/issn/1941-6016).
17. Rosenblatt, Frank (1961-03-15). *DTIC AD0256582: PRINCIPLES OF NEURODYNAMICS. PERCEPTRONS AND THE THEORY OF BRAIN MECHANISMS* (https://archive.org/details/DTIC_AD0256582/page/n3/mode/2up). Defense Technical Information Center.
18. F. Rosenblatt, "Perceptual Generalization over Transformation Groups", pp. 63--100 in *Self-organizing Systems: Proceedings of an Inter-disciplinary Conference, 5 and 6 May 1959*. Edited by Marshall C. Yovitz and Scott Cameron. London, New York, [etc.], Pergamon Press, 1960. ix, 322 p.
19. Nakano, Kaoru (1971). "Learning Process in a Model of Associative Memory". *Pattern Recognition and Machine Learning*. pp. 172–186. doi:10.1007/978-1-4615-7566-5_15 (https://doi.org/10.1007%2F978-1-4615-7566-5_15). ISBN 978-1-4615-7568-9.
20. Nakano, Kaoru (1972). "Associatron-A Model of Associative Memory". *IEEE Transactions on Systems, Man, and Cybernetics*. SMC-2 (3): 380–388. Bibcode:1972ITSMC...2..380N (https://ui.adsabs.harvard.edu/abs/1972ITSMC...2..380N). doi:10.1109/TSMC.1972.4309133 (https://doi.org/10.1109%2FTSMC.1972.4309133).
21. Amari, Shun-Ichi (1972). "Learning patterns and pattern sequences by self-organizing nets of threshold elements". *IEEE Transactions*. **C** (21): 1197–1206.
22. Little, W. A. (1974). "The Existence of Persistent States in the Brain". *Mathematical Biosciences*. **19** (1–2): 101–120. doi:10.1016/0025-5564(74)90031-5 (https://doi.org/10.1016%2F0025-5564%2874%2990031-5).
23. Lenz, W. (1920), "Beiträge zum Verständnis der magnetischen Eigenschaften in festen Körpern", *Physikalische Zeitschrift*, **21**: 613–615.
24. Ising, E. (1925), "Beitrag zur Theorie des Ferromagnetismus", *Z. Phys.*, **31** (1): 253–258, Bibcode:1925ZPhy...31..253I (https://ui.adsabs.harvard.edu/abs/1925ZPhy...31..253I), doi:10.1007/BF02980577 (https://doi.org/10.1007%2FBF02980577), S2CID 122157319 (https://api.semanticscholar.org/CorpusID:122157319)
25. Brush, Stephen G. (1967). "History of the Lenz-Ising Model". *Reviews of Modern Physics*. **39** (4): 883–893. Bibcode:1967RvMP...39..883B (https://ui.adsabs.harvard.edu/abs/1967RvMP...39..883B). doi:10.1103/RevModPhys.39.883 (https://doi.org/10.1103%2FRevModPhys.39.883).
26. Glauber, Roy J. (February 1963). "Roy J. Glauber "Time-Dependent Statistics of the Ising Model" " (https://aip.scitation.org/doi/abs/10.1063/1.1703954). *Journal of Mathematical Physics*. **4** (2): 294–307. doi:10.1063/1.1703954 (https://doi.org/10.1063%2F1.1703954). Retrieved 2021-03-21.
27. Sherrington, David; Kirkpatrick, Scott (1975-12-29). "Solvable Model of a Spin-Glass" (https://link.aps.org/doi/10.1103/PhysRevLett.35.1792). *Physical Review Letters*. **35** (26): 1792–1796. Bibcode:1975PhRvL..35.1792S (https://ui.adsabs.harvard.edu/abs/1975PhRvL..35.1792S). doi:10.1103/PhysRevLett.35.1792 (https://doi.org/10.1103%2FPhysRevLett.35.1792). ISSN 0031-9007 (https://search.worldcat.org/issn/0031-9007).
28. Hopfield, J. J. (1982). "Neural networks and physical systems with emergent collective computational abilities" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC346238). *Proceedings of the National Academy of Sciences*. **79** (8): 2554–2558. Bibcode:1982PNAS...79.2554H (https://ui.adsabs.harvard.edu/abs/1982PNAS...79.2554H). doi:10.1073/pnas.79.8.2554 (https://doi.org/10.1073%2Fpnas.79.8.2554). PMC 346238 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC346238). PMID 6953413 (https://pubmed.ncbi.nlm.nih.gov/6953413).

29. Hopfield, J. J. (1984). "Neurons with graded response have collective computational properties like those of two-state neurons" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC345226). *Proceedings of the National Academy of Sciences*. **81** (10): 3088–3092. Bibcode:1984PNAS...81.3088H (https://ui.adsabs.harvard.edu/abs/1984PNAS...81.3088H). doi:10.1073/pnas.81.10.3088 (https://doi.org/10.1073%2Fpnas.81.10.3088). PMC 345226 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC345226). PMID 6587342 (https://pubmed.ncbi.nlm.nih.gov/6587342).
30. Engel, A.; Broeck, C. van den (2001). *Statistical mechanics of learning*. Cambridge, UK; New York, NY: Cambridge University Press. ISBN 978-0-521-77307-2.
31. Seung, H. S.; Sompolinsky, H.; Tishby, N. (1992-04-01). "Statistical mechanics of learning from examples" (https://journals.aps.org/pra/abstract/10.1103/PhysRevA.45.6056). *Physical Review A*. **45** (8): 6056–6091. Bibcode:1992PhRvA..45.6056S (https://ui.adsabs.harvard.edu/abs/1992PhRvA..45.6056S). doi:10.1103/PhysRevA.45.6056 (https://doi.org/10.1103%2FPhysRevA.45.6056). PMID 9907706 (https://pubmed.ncbi.nlm.nih.gov/9907706).
32. Jordan, Michael I. (May 1986). Serial Order: A Parallel Distributed Processing Approach (https://cseweb.ucsd.edu/~gary/PAPER-SUGGESTIONS/Jordan-TR-8604.pdf) (PDF) (Report). Institute for Cognitive Science, University of California, San Diego.
33. Elman, Jeffrey L. (1990). "Finding Structure in Time". *Cognitive Science*. **14** (2): 179–211. doi:10.1207/s15516709cog1402_1 (https://doi.org/10.1207%2Fs15516709cog1402_1).
34. Hochreiter, Sepp; Schmidhuber, Jürgen (1997). "Long Short-Term Memory". *Neural Computation*. **9** (8): 1735–1780. doi:10.1162/neco.1997.9.8.1735 (https://doi.org/10.1162%2Fneco.1997.9.8.1735). PMID 9377276 (https://pubmed.ncbi.nlm.nih.gov/9377276).
35. Schuster, Mike; Paliwal, Kuldip K. (1997). "Bidirectional recurrent neural networks". *IEEE Transactions on Signal Processing*. **45** (11): 2673–2681. doi:10.1109/78.650093 (https://doi.org/10.1109%2F78.650093).
36. Graves, Alex; Schmidhuber, Jürgen (2005). "Framewise phoneme classification with bidirectional LSTM and other neural network architectures". *Neural Networks*. **18** (5–6): 602–610. doi:10.1016/j.neunet.2005.06.042 (https://doi.org/10.1016%2Fj.neunet.2005.06.042). PMID 16112549 (https://pubmed.ncbi.nlm.nih.gov/16112549).
37. Fan, Bo; Wang, Lijuan; Soong, Frank K.; Xie, Lei (2015). "Photo-Real Talking Head with Deep Bidirectional LSTM". *Proceedings of ICASSP 2015 IEEE International Conference on Acoustics, Speech and Signal Processing*. pp. 4884–8. doi:10.1109/ICASSP.2015.7178899 (https://doi.org/10.1109%2FICASSP.2015.7178899). ISBN 978-1-4673-6997-8.
38. Sak, Haşim; Senior, Andrew; Rao, Kanishka; Beaufays, Françoise; Schalkwyk, Johan (September 2015). "Google voice search: faster and more accurate" (http://googleresearch.blogspot.ch/2015/09/google-voice-search-faster-and-more.html).
39. Mikolov, Tomáš; Karafiát, Martin; Burget, Lukáš; Černocký, Jan; Khudanpur, Sanjeev (2010). *Recurrent neural network based language model* (https://www.isca-archive.org/interspeech_2010/mikolov10_interspeech.html). Interspeech 2010. pp. 1045–1048. doi:10.21437/Interspeech.2010-343 (https://doi.org/10.21437%2FInterspeech.2010-343).
40. Cho, Kyunghyun; van Merriënboer, Bart; Gulcehre, Caglar; Bahdanau, Dzmitry; Bougares, Fethi; Schwenk, Holger; Bengio, Yoshua (2014). "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation" (https://aclanthology.org/D14-1179/). *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Doha, Qatar: Association for Computational Linguistics. pp. 1724–1734. doi:10.3115/v1/D14-1179 (https://doi.org/10.3115%2Fv1%2FD14-1179).
41. Sutskever, Ilya; Vinyals, Oriol; Le, Quoc V. (2014). "Sequence to Sequence Learning with Neural Networks" (https://proceedings.neurips.cc/paper/5346-sequence-to-sequence-learning-with-neural-). *Advances in Neural Information Processing Systems*. Vol. 27. pp. 3104–3112.

42. Peters ME, Neumann M, Iyyer M, Gardner M, Clark C, Lee K, Zettlemoyer L (2018). "Deep contextualized word representations". arXiv:1802.05365 (https://arxiv.org/abs/1802.05365) [cs.CL (https://arxiv.org/archive/cs.CL)].
43. Vaswani, Ashish; Shazeer, Noam; Parmar, Niki; Uszkoreit, Jakob; Jones, Llion; Gomez, Aidan N; Kaiser, Ł ukasz; Polosukhin, Illia (2017). "Attention is All you Need" (https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html). *Advances in Neural Information Processing Systems*. **30**. Curran Associates, Inc.
44. Oord, Aäron van den; Kalchbrenner, Nal; Kavukcuoglu, Koray (2016-06-11). "Pixel Recurrent Neural Networks" (https://proceedings.mlr.press/v48/oord16.html). *Proceedings of the 33rd International Conference on Machine Learning*. PMLR: 1747–1756.
45. Cruse, Holk; *Neural Networks as Cybernetic Systems* (http://www.brains-minds-media.org/archive/615/bmm615.pdf), 2nd and revised edition
46. Elman, Jeffrey L. (1990). "Finding Structure in Time" (https://doi.org/10.1016%2F0364-0213%2890%2990002-E). *Cognitive Science*. **14** (2): 179–211. doi:10.1016/0364-0213(90)90002-E (https://doi.org/10.1016%2F0364-0213%2890%2990002-E).
47. Jordan, Michael I. (1997-01-01). "Serial Order: A Parallel Distributed Processing Approach". *Neural-Network Models of Cognition — Biobehavioral Foundations*. Advances in Psychology. Vol. 121. pp. 471–495. doi:10.1016/s0166-4115(97)80111-2 (https://doi.org/10.1016%2Fs0166-4115%2897%2980111-2). ISBN 978-0-444-81931-4. S2CID 15375627 (https://api.semanticscholar.org/CorpusID:15375627).
48. Gers, Felix A.; Schraudolph, Nicol N.; Schmidhuber, Jürgen (2002). "Learning Precise Timing with LSTM Recurrent Networks" (http://www.jmlr.org/papers/volume3/gers02a/gers02a.pdf) (PDF). *Journal of Machine Learning Research*. **3**: 115–143. Retrieved 2017-06-13.
49. Hochreiter, Sepp (1991). *Untersuchungen zu dynamischen neuronalen Netzen* (http://people.idsia.ch/~juergen/SeppHochreiter1991ThesisAdvisorSchmidhuber.pdf) (PDF) (Diploma). Institut f. Informatik, Technical University of Munich.
50. Bayer, Justin; Wierstra, Daan; Togelius, Julian; Schmidhuber, Jürgen (2009-09-14). "Evolving Memory Cell Structures for Sequence Learning". *Artificial Neural Networks – ICANN 2009* (https://mediatum.ub.tum.de/doc/1289041/document.pdf) (PDF). Lecture Notes in Computer Science. Vol. 5769. Berlin, Heidelberg: Springer. pp. 755–764. doi:10.1007/978-3-642-04277-5_76 (https://doi.org/10.1007%2F978-3-642-04277-5_76). ISBN 978-3-642-04276-8.
51. Fernández, Santiago; Graves, Alex; Schmidhuber, Jürgen (2007). "Sequence labelling in structured domains with hierarchical recurrent neural networks" (https://www.ijcai.org/Proceedings/07/Papers/124.pdf) (PDF). *Proceedings of the 20th International Joint Conference on Artificial Intelligence, Ijcai 2007*. pp. 774–9.
52. Gers, Felix A.; Schmidhuber, Jürgen (2001). "LSTM Recurrent Networks Learn Simple Context Free and Context Sensitive Languages" (https://web.archive.org/web/20170706014426/ftp://ftp.idsia.ch/pub/juergen/L-IEEE.pdf) (PDF). *IEEE Transactions on Neural Networks*. **12** (6): 1333–40. Bibcode:2001ITNN...12.1333G (https://ui.adsabs.harvard.edu/abs/2001ITNN...12.1333G). doi:10.1109/72.963769 (https://doi.org/10.1109%2F72.963769). PMID 18249962 (https://pubmed.ncbi.nlm.nih.gov/18249962). S2CID 10192330 (https://api.semanticscholar.org/CorpusID:10192330). Archived from the original (ftp://ftp.idsia.ch/pub/juergen/L-IEEE.pdf) (PDF) on 2017-07-06. Retrieved 2017-12-12.
53. Heck, Joel; Salem, Fathi M. (2017-01-12). "Simplified Minimal Gated Unit Variations for Recurrent Neural Networks". arXiv:1701.03452 (https://arxiv.org/abs/1701.03452) [cs.NE (https://arxiv.org/archive/cs.NE)].
54. Dey, Rahul; Salem, Fathi M. (2017-01-20). "Gate-Variants of Gated Recurrent Unit (GRU) Neural Networks". arXiv:1701.05923 (https://arxiv.org/abs/1701.05923) [cs.NE (https://arxiv.org/archive/cs.NE)].

55. Britz, Denny (October 27, 2015). "Recurrent Neural Network Tutorial, Part 4 – Implementing a GRU/LSTM RNN with Python and Theano – WildML" (http://www.wildml.com/2015/10/recurrent-neural-network-tutorial-part-4-implementing-a-grulstm-rnn-with-python-and-theano/). *Wildml.com*. Retrieved May 18, 2016.
56. Chung, Junyoung; Gulcehre, Caglar; Cho, KyungHyun; Bengio, Yoshua (2014). "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling". arXiv:1412.3555 (https://arxiv.org/abs/1412.3555) [cs.NE (https://arxiv.org/archive/cs.NE)].
57. Gruber, N.; Jockisch, A. (2020), "Are GRU cells more specific and LSTM cells more sensitive in motive classification of text?", *Frontiers in Artificial Intelligence*, **3** 40, doi:10.3389/frai.2020.00040 (https://doi.org/10.3389%2Ffrai.2020.00040), PMC 7861254 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7861254), PMID 33733157 (https://pubmed.ncbi.nlm.nih.gov/33733157), S2CID 220252321 (https://api.semanticscholar.org/CorpusID:220252321)
58. Kosko, Bart (1988). "Bidirectional associative memories" (https://archive.org/details/ieee-trans-on-systems-man-and-cybernetics_january-february-1988_18_1/page/49). *IEEE Transactions on Systems, Man, and Cybernetics*. **18** (1): 49–60. Bibcode:1988ITSMC..18...49K (https://ui.adsabs.harvard.edu/abs/1988ITSMC..18...49K). doi:10.1109/21.87054 (https://doi.org/10.1109%2F21.87054). S2CID 59875735 (https://api.semanticscholar.org/CorpusID:59875735).
59. Rakkiyappan, Rajan; Chandrasekar, Arunachalam; Lakshmanan, Subramanian; Park, Ju H. (2 January 2015). "Exponential stability for markovian jumping stochastic BAM neural networks with mode-dependent probabilistic time-varying delays and impulse control". *Complexity*. **20** (3): 39–65. Bibcode:2015Cmplx..20c..39R (https://ui.adsabs.harvard.edu/abs/2015Cmplx..20c..39R). doi:10.1002/cplx.21503 (https://doi.org/10.1002%2Fcplx.21503).
60. Rojas, Rául (1996). *Neural networks: a systematic introduction* (https://books.google.com/books?id=txsjjYzFJS4C&pg=PA336). Springer. p. 336. ISBN 978-3-540-60505-8.
61. Jaeger, Herbert; Haas, Harald (2004-04-02). "Harnessing Nonlinearity: Predicting Chaotic Systems and Saving Energy in Wireless Communication". *Science*. **304** (5667): 78–80. Bibcode:2004Sci...304...78J (https://ui.adsabs.harvard.edu/abs/2004Sci...304...78J). doi:10.1126/science.1091277 (https://doi.org/10.1126%2Fscience.1091277). PMID 15064413 (https://pubmed.ncbi.nlm.nih.gov/15064413). S2CID 2184251 (https://api.semanticscholar.org/CorpusID:2184251).
62. Maass, Wolfgang; Natschläger, Thomas; Markram, Henry (2002). "Real-time computing without stable states: a new framework for neural computation based on perturbations" (https://igi-web.tugraz.at/people/maass/psfiles/130.pdf) (PDF). *Neural Computation*. **14** (11): 2531–2560. doi:10.1162/089976602760407955 (https://doi.org/10.1162%2F089976602760407955). PMID 12433288 (https://pubmed.ncbi.nlm.nih.gov/12433288). S2CID 1045112 (https://api.semanticscholar.org/CorpusID:1045112).
63. Goller, Christoph; Küchler, Andreas (1996). "Learning task-dependent distributed representations by backpropagation through structure". *Proceedings of International Conference on Neural Networks (ICNN'96)*. Vol. 1. p. 347. doi:10.1109/ICNN.1996.548916 (https://doi.org/10.1109%2FICNN.1996.548916). ISBN 978-0-7803-3210-2. S2CID 6536466 (https://api.semanticscholar.org/CorpusID:6536466).
64. Linnainmaa, Seppo (1970). *The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors* (MSc) (in Finnish). University of Helsinki.
65. Griewank, Andreas; Walther, Andrea (2008). *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation* (https://books.google.com/books?id=xoiiLaRxcbEC) (Second ed.). SIAM. ISBN 978-0-89871-776-1.
66. Socher, Richard; Lin, Cliff; Ng, Andrew Y.; Manning, Christopher D., "Parsing Natural Scenes and Natural Language with Recursive Neural Networks" (https://ai.stanford.edu/~ang/papers/icml11-ParsingWithRecursiveNeuralNetworks.pdf) (PDF), *28th International Conference on Machine Learning (ICML 2011)*

67. Socher, Richard; Perelygin, Alex; Wu, Jean Y.; Chuang, Jason; Manning, Christopher D.; Ng, Andrew Y.; Potts, Christopher. "Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank" (http://nlp.stanford.edu/~socherr/EMNLP2013_RNTN.pdf) (PDF). *Emnlp 2013*.
68. Graves, Alex; Wayne, Greg; Danihelka, Ivo (2014). "Neural Turing Machines". arXiv:1410.5401 (https://arxiv.org/abs/1410.5401) [cs.NE (https://arxiv.org/archive/cs.NE)].
69. Graves, Alex; Wayne, Greg; Reynolds, Malcolm; Harley, Tim; Danihelka, Ivo; Grabska-Barwińska, Agnieszka; Colmenarejo, Sergio Gómez; Grefenstette, Edward; Ramalho, Tiago (2016-10-12). "Hybrid computing using a neural network with dynamic external memory" (http://www.nature.com/articles/nature20101.epdf?author_access_token=ImTXBI8aWbYxYQ51Plys8NRgN0jAjWel9jnR3ZoTv0MggmpDmwljGswxVdeocYSurJ3hxupzWuRNeGvvXnoO8o4jTJcnAyhGuZzXJ1GEaD-Z7E6X_a9R-xqJ9TfJWBqz). *Nature*. **538** (7626): 471–476. Bibcode:2016Natur.538..471G (https://ui.adsabs.harvard.edu/abs/2016Natur.538..471G). doi:10.1038/nature20101 (https://doi.org/10.1038%2Fnature20101). ISSN 1476-4687 (https://search.worldcat.org/issn/1476-4687). PMID 27732574 (https://pubmed.ncbi.nlm.nih.gov/27732574). S2CID 205251479 (https://api.semanticscholar.org/CorpusID:205251479).
70. Sun, Guo-Zheng; Giles, C. Lee; Chen, Hsing-Hen (1998). "The Neural Network Pushdown Automaton: Architecture, Dynamics and Training". In Giles, C. Lee; Gori, Marco (eds.). *Adaptive Processing of Sequences and Data Structures*. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer. pp. 296–345. doi:10.1007/bfb0054003 (https://doi.org/10.1007%2Fbfb0054003). ISBN 978-3-540-64341-8.
71. Hyötyniemi, Heikki (1996). "Turing machines are recurrent neural networks". *Proceedings of STeP '96/Publications of the Finnish Artificial Intelligence Society*: 13–24.
72. Robinson, Anthony J.; Fallside, Frank (1987). *The Utility Driven Dynamic Error Propagation Network* (https://books.google.com/books?id=6JYYMwEACAAJ). Technical Report CUED/F-INFENG/TR.1. Department of Engineering, University of Cambridge.
73. Williams, Ronald J.; Zipser, D. (1 February 2013). "Gradient-based learning algorithms for recurrent networks and their computational complexity". In Chauvin, Yves; Rumelhart, David E. (eds.). *Backpropagation: Theory, Architectures, and Applications* (https://books.google.com/books?id=B71nu3LDpREC). Psychology Press. ISBN 978-1-134-77581-1.
74. Schmidhuber, Jürgen (1989-01-01). "A Local Learning Algorithm for Dynamic Feedforward and Recurrent Networks". *Connection Science*. **1** (4): 403–412. doi:10.1080/09540098908915650 (https://doi.org/10.1080%2F09540098908915650). S2CID 18721007 (https://api.semanticscholar.org/CorpusID:18721007).
75. Príncipe, José C.; Euliano, Neil R.; Lefebvre, W. Curt (2000). *Neural and adaptive systems: fundamentals through simulations* (https://books.google.com/books?id=jgMZAQAAIAAJ). Wiley. ISBN 978-0-471-35167-2.
76. Yann, Ollivier; Tallec, Corentin; Charpiat, Guillaume (2015-07-28). "Training recurrent networks online without backtracking". arXiv:1507.07680 (https://arxiv.org/abs/1507.07680) [cs.NE (https://arxiv.org/archive/cs.NE)].
77. Schmidhuber, Jürgen (1992-03-01). "A Fixed Size Storage O(n3) Time Complexity Learning Algorithm for Fully Recurrent Continually Running Networks". *Neural Computation*. **4** (2): 243–248. doi:10.1162/neco.1992.4.2.243 (https://doi.org/10.1162%2Fneco.1992.4.2.243). S2CID 11761172 (https://api.semanticscholar.org/CorpusID:11761172).
78. Williams, Ronald J. (1989). Complexity of exact gradient computation algorithms for recurrent neural networks (https://web.archive.org/web/20171020033840/http://citeseerx.ist.psu.edu/showciting?cid=128036) (Report). Technical Report NU-CCS-89-27. Boston (MA): Northeastern University, College of Computer Science. Archived from the original (http://citeseerx.ist.psu.edu/showciting?cid=128036) on 2017-10-20. Retrieved 2017-07-02.

79. Pearlmutter, Barak A. (1989-06-01). "Learning State Space Trajectories in Recurrent Neural Networks" (http://repository.cmu.edu/cgi/viewcontent.cgi?article=2865&context=compsci). *Neural Computation*. **1** (2): 263–269. doi:10.1162/neco.1989.1.2.263 (https://doi.org/10.1162%2Fneco.1989.1.2.263). S2CID 16813485 (https://api.semanticscholar.org/CorpusID:16813485).
80. Hochreiter, Sepp; et al. (15 January 2001). "Gradient flow in recurrent nets: the difficulty of learning long-term dependencies" (https://books.google.com/books?id=NWOcMVA64aAC). In Kolen, John F.; Kremer, Stefan C. (eds.). *A Field Guide to Dynamical Recurrent Networks*. John Wiley & Sons. ISBN 978-0-7803-5369-5.
81. Hochreiter, Sepp; Schmidhuber, Jürgen (1997-11-01). "Long Short-Term Memory". *Neural Computation*. **9** (8): 1735–1780. doi:10.1162/neco.1997.9.8.1735 (https://doi.org/10.1162%2Fneco.1997.9.8.1735). PMID 9377276 (https://pubmed.ncbi.nlm.nih.gov/9377276). S2CID 1915014 (https://api.semanticscholar.org/CorpusID:1915014).
82. Li, Shuai; Li, Wanqing; Cook, Chris; Zhu, Ce; Yanbo, Gao (2018). "Independently Recurrent Neural Network (IndRNN): Building a Longer and Deeper RNN". arXiv:1803.04831 (https://arxiv.org/abs/1803.04831) [cs.CV (https://arxiv.org/archive/cs.CV)].
83. Campolucci, Paolo; Uncini, Aurelio; Piazza, Francesco; Rao, Bhaskar D. (1999). "On-Line Learning Algorithms for Locally Recurrent Neural Networks". *IEEE Transactions on Neural Networks*. **10** (2): 253–271. Bibcode:1999ITNN...10..253C (https://ui.adsabs.harvard.edu/abs/1999ITNN...10..253C). doi:10.1109/72.750549 (https://doi.org/10.1109%2F72.750549). PMID 18252525 (https://pubmed.ncbi.nlm.nih.gov/18252525).
84. Wan, Eric A.; Beaufays, Françoise (1996). "Diagrammatic derivation of gradient algorithms for neural networks" (https://archive.org/details/sim_neural-computation_1996-01_8_1/page/182). *Neural Computation*. **8**: 182–201. doi:10.1162/neco.1996.8.1.182 (https://doi.org/10.1162%2Fneco.1996.8.1.182). S2CID 15512077 (https://api.semanticscholar.org/CorpusID:15512077).
85. Campolucci, Paolo; Uncini, Aurelio; Piazza, Francesco (2000). "A Signal-Flow-Graph Approach to On-line Gradient Calculation". *Neural Computation*. **12** (8): 1901–1927. doi:10.1162/089976600300015196 (https://doi.org/10.1162%2F089976600300015196). PMID 10953244 (https://pubmed.ncbi.nlm.nih.gov/10953244). S2CID 15090951 (https://api.semanticscholar.org/CorpusID:15090951).
86. Graves, Alex; Fernández, Santiago; Gomez, Faustino J. (2006). "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks" (https://axon.cs.byu.edu/~martinez/classes/778/Papers/p369-graves.pdf) (PDF). *Proceedings of the International Conference on Machine Learning*. pp. 369–376. doi:10.1145/1143844.1143891 (https://doi.org/10.1145%2F1143844.1143891). ISBN 1-59593-383-2.
87. Hannun, Awni (2017-11-27). "Sequence Modeling with CTC" (https://distill.pub/2017/ctc). *Distill*. **2** (11) e8. doi:10.23915/distill.00008 (https://doi.org/10.23915%2Fdistill.00008). ISSN 2476-0757 (https://search.worldcat.org/issn/2476-0757).
88. Gomez, Faustino J.; Miikkulainen, Risto (1999), "Solving non-Markovian control tasks with neuroevolution" (http://www.cs.utexas.edu/users/nn/downloads/papers/gomez.ijcai99.pdf) (PDF), *IJCAI 99*, Morgan Kaufmann, retrieved 5 August 2017
89. Syed, Omar (May 1995). *Applying Genetic Algorithms to Recurrent Neural Networks for Learning Network Parameters and Architecture* (http://arimaa.com/arimaa/about/Thesis/) (MSc). Department of Electrical Engineering, Case Western Reserve University.
90. Gomez, Faustino J.; Schmidhuber, Jürgen; Miikkulainen, Risto (June 2008). "Accelerated Neural Evolution Through Cooperatively Coevolved Synapses" (https://www.jmlr.org/papers/volume9/gomez08a/gomez08a.pdf) (PDF). *Journal of Machine Learning Research*. **9**: 937–965.

91. Schmidhuber, Jürgen (1992). "Learning complex, extended sequences using the principle of history compression" (https://web.archive.org/web/20170706014739/ftp://ftp.idsia.ch/pub/juergen/chunker.pdf) (PDF). *Neural Computation*. **4** (2): 234–242. doi:10.1162/neco.1992.4.2.234 (https://doi.org/10.1162%2Fneco.1992.4.2.234). S2CID 18271205 (https://api.semanticscholar.org/CorpusID:18271205). Archived from the original (ftp://ftp.idsia.ch/pub/juergen/chunker.pdf) (PDF) on 2017-07-06.
92. Schmidhuber, Jürgen (2015). "Deep Learning" (https://doi.org/10.4249%2Fscholarpedia.32832). *Scholarpedia*. **10** (11) 32832. Bibcode:2015SchpJ..1032832S (https://ui.adsabs.harvard.edu/abs/2015SchpJ..1032832S). doi:10.4249/scholarpedia.32832 (https://doi.org/10.4249%2Fscholarpedia.32832).
93. Schmidhuber, Jürgen (1993). *Habilitation thesis: System modeling and optimization* (ftp://ftp.idsia.ch/pub/juergen/habilitation.pdf) (PDF). Page 150 ff demonstrates credit assignment across the equivalent of 1,200 layers in an unfolded RNN.
94. Giles, C. Lee; Miller, Clifford B.; Chen, Dong; Chen, Hsing-Hen; Sun, Guo-Zheng; Lee, Yee-Chun (1992). "Learning and Extracting Finite State Automata with Second-Order Recurrent Neural Networks" (https://clgiles.ist.psu.edu/pubs/NC1992-recurrent-NN.pdf) (PDF). *Neural Computation*. **4** (3): 393–405. doi:10.1162/neco.1992.4.3.393 (https://doi.org/10.1162%2Fneco.1992.4.3.393). S2CID 19666035 (https://api.semanticscholar.org/CorpusID:19666035).
95. Omlin, Christian W.; Giles, C. Lee (1996). "Constructing Deterministic Finite-State Automata in Recurrent Neural Networks". *Journal of the ACM*. **45** (6): 937–972. doi:10.1145/235809.235811 (https://doi.org/10.1145%2F235809.235811). S2CID 228941 (https://api.semanticscholar.org/CorpusID:228941).
96. Paine, Rainer W.; Tani, Jun (2005-09-01). "How Hierarchical Control Self-organizes in Artificial Adaptive Systems". *Adaptive Behavior*. **13** (3): 211–225. doi:10.1177/105971230501300303 (https://doi.org/10.1177%2F105971230501300303). S2CID 9932565 (https://api.semanticscholar.org/CorpusID:9932565).
97. "Burns, Benureau, Tani (2018) A Bergson-Inspired Adaptive Time Constant for the Multiple Timescales Recurrent Neural Network Model. JNNS" (https://www.researchgate.net/publication/328474302).
98. Barkan, Oren; Benchimol, Jonathan; Caspi, Itamar; Cohen, Eliya; Hammer, Allon; Koenigstein, Noam (2023). "Forecasting CPI inflation components with Hierarchical Recurrent Neural Networks". *International Journal of Forecasting*. **39** (3): 1145–1162. arXiv:2011.07920 (https://arxiv.org/abs/2011.07920). doi:10.1016/j.ijforecast.2022.04.009 (https://doi.org/10.1016%2Fj.ijforecast.2022.04.009).
99. Tutschku, Kurt (June 1995). *Recurrent Multilayer Perceptrons for Identification and Control: The Road to Applications*. Institute of Computer Science Research Report. Vol. 118. University of Würzburg Am Hubland. CiteSeerX 10.1.1.45.3527 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.45.3527). `{{cite book}}`: Cite uses deprecated parameter `|citeseerx=` (help)
100. Yamashita, Yuichi; Tani, Jun (2008-11-07). "Emergence of Functional Hierarchy in a Multiple Timescale Neural Network Model: A Humanoid Robot Experiment" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC2570613). *PLOS Computational Biology*. **4** (11) e1000220. Bibcode:2008PLSCB...4E0220Y (https://ui.adsabs.harvard.edu/abs/2008PLSCB...4E0220Y). doi:10.1371/journal.pcbi.1000220 (https://doi.org/10.1371%2Fjournal.pcbi.1000220). PMC 2570613 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC2570613). PMID 18989398 (https://pubmed.ncbi.nlm.nih.gov/18989398).
101. Alnajjar, Fady; Yamashita, Yuichi; Tani, Jun (2013). "The hierarchical and functional connectivity of higher-order cognitive mechanisms: neurorobotic model to investigate the stability and flexibility of working memory" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3575058). *Frontiers in Neurorobotics*. **7**: 2. doi:10.3389/fnbot.2013.00002 (https://doi.org/10.3389%2Ffnbot.2013.00002). PMC 3575058 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3575058). PMID 23423881 (https://pubmed.ncbi.nlm.nih.gov/23423881).

102. "Proceedings of the 28th Annual Conference of the Japanese Neural Network Society (October, 2018)" (https://web.archive.org/web/20200509004753/http://jnns.org/conference/2018/JNNS2018_Technical_Programs.pdf) (PDF). Archived from the original (http://jnns.org/conference/2018/JNNS2018_Technical_Programs.pdf) (PDF) on 2020-05-09. Retrieved 2021-02-06.
103. Snider, Greg (2008), "Cortical computing with memristive nanodevices" (https://web.archive.org/web/20160516070906/http://www.scidacreview.org/0804/html/hardware.html), *Sci-DAC Review*, **10**: 58–65, archived from the original (http://www.scidacreview.org/0804/html/hardware.html) on 2016-05-16, retrieved 2019-09-06
104. Caravelli, Francesco; Traversa, Fabio Lorenzo; Di Ventra, Massimiliano (2017). "The complex dynamics of memristive circuits: analytical results and universal slow relaxation". *Physical Review E*. **95** (2) 022140. arXiv:1608.08651 (https://arxiv.org/abs/1608.08651). Bibcode:2017PhRvE..95b2140C (https://ui.adsabs.harvard.edu/abs/2017PhRvE..95b2140C). doi:10.1103/PhysRevE.95.022140 (https://doi.org/10.1103%2FPhysRevE.95.022140). PMID 28297937 (https://pubmed.ncbi.nlm.nih.gov/28297937). S2CID 6758362 (https://api.semanticscholar.org/CorpusID:6758362).
105. Harvey, Inman; Husbands, Phil; Cliff, Dave (1994), "Seeing the light: Artificial evolution, real vision" (https://www.researchgate.net/publication/229091538_Seeing_the_Light_Artificial_Evolution_Real_Vision), *3rd international conference on Simulation of adaptive behavior: from animals to animats 3*, pp. 392–401
106. Quinn, Matt (2001). "Evolving communication without dedicated communication channels". *Advances in Artificial Life: 6th European Conference, ECAL 2001*. pp. 357–366. doi:10.1007/3-540-44811-X_38 (https://doi.org/10.1007%2F3-540-44811-X_38). ISBN 978-3-540-42567-0.
107. Beer, Randall D. (1997). "The dynamics of adaptive behavior: A research program" (https://archive.org/details/sim_robotics-and-autonomous-systems_1997-06_20_2-4/page/257). *Robotics and Autonomous Systems*. **20** (2–4): 257–289. doi:10.1016/S0921-8890(96)00063-2 (https://doi.org/10.1016%2FS0921-8890%2896%2900063-2).
108. Sherstinsky, Alex (2018-12-07). Bloem-Reddy, Benjamin; Paige, Brooks; Kusner, Matt; Caruana, Rich; Rainforth, Tom; Teh, Yee Whye (eds.). *Deriving the Recurrent Neural Network Definition and RNN Unrolling Using Signal Processing* (https://www.researchgate.net/publication/331718291). Critiquing and Correcting Trends in Machine Learning Workshop at NeurIPS-2018 (https://ml-critique-correct.github.io/).
109. Siegelmann, Hava T.; Horne, Bill G.; Giles, C. Lee (1995). "Computational Capabilities of Recurrent NARX Neural Networks" (https://books.google.com/books?id=830-HAAACAAJ&pg=PA208). *IEEE Transactions on Systems, Man, and Cybernetics - Part B: Cybernetics*. **27** (2): 208–15. doi:10.1109/3477.558801 (https://doi.org/10.1109%2F3477.558801). PMID 18255858 (https://pubmed.ncbi.nlm.nih.gov/18255858).
110. Miljanovic, Milos (Feb–Mar 2012). "Comparative analysis of Recurrent and Finite Impulse Response Neural Networks in Time Series Prediction" (http://www.ijcse.com/docs/INDJCSE12-03-01-028.pdf) (PDF). *Indian Journal of Computer and Engineering*. **3** (1).
111. Hodassman, Shiri; Meir, Yuval; Kisos, Karin; Ben-Noam, Itamar; Tugendhaft, Yael; Goldental, Amir; Vardi, Roni; Kanter, Ido (2022-09-29). "Brain inspired neuronal silencing mechanism to enable reliable sequence identification" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9523036). *Scientific Reports*. **12** (1): 16003. arXiv:2203.13028 (https://arxiv.org/abs/2203.13028). Bibcode:2022NatSR..1216003H (https://ui.adsabs.harvard.edu/abs/2022NatSR..1216003H). doi:10.1038/s41598-022-20337-x (https://doi.org/10.1038%2Fs41598-022-20337-x). ISSN 2045-2322 (https://search.worldcat.org/issn/2045-2322). PMC 9523036 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9523036). PMID 36175466 (https://pubmed.ncbi.nlm.nih.gov/36175466).
112. Metz, Cade (May 18, 2016). "Google Built Its Very Own Chips to Power Its AI Bots" (https://www.wired.com/2016/05/google-tpu-custom-chips/). *Wired*.

113. Sutskever, Ilya; Vinyals, Oriol; Le, Quoc V. (2014). "Sequence to Sequence Learning with Neural Networks" (https://papers.nips.cc/paper/5346-sequence-to-sequence-learning-with-neural-networks.pdf) (PDF). *Electronic Proceedings of the Neural Information Processing Systems Conference*. **27**: 5346. arXiv:1409.3215 (https://arxiv.org/abs/1409.3215). Bibcode:2014arXiv1409.3215S (https://ui.adsabs.harvard.edu/abs/2014arXiv1409.3215S).
114. Mayer, Hermann; Gomez, Faustino J.; Wierstra, Daan; Nagy, Istvan; Knoll, Alois; Schmidhuber, Jürgen (October 2006). "A System for Robotic Heart Surgery that Learns to Tie Knots Using Recurrent Neural Networks". *2006 IEEE/RSJ International Conference on Intelligent Robots and Systems*. pp. 543–548. doi:10.1109/IROS.2006.282190 (https://doi.org/10.1109%2FIROS.2006.282190). ISBN 978-1-4244-0258-8. S2CID 12284900 (https://api.semanticscholar.org/CorpusID:12284900).
115. Wierstra, Daan; Schmidhuber, Jürgen; Gomez, Faustino J. (2005). "Evolino: Hybrid Neuroevolution/Optimal Linear Search for Sequence Learning" (https://www.academia.edu/5830256). *Proceedings of the 19th International Joint Conference on Artificial Intelligence (IJCAI), Edinburgh*. pp. 853–8. OCLC 62330637 (https://search.worldcat.org/oclc/62330637).
116. Petneházi, Gábor (2019-01-01). "Recurrent neural networks for time series forecasting". arXiv:1901.00069 (https://arxiv.org/abs/1901.00069) [cs.LG (https://arxiv.org/archive/cs.LG)].
117. Hewamalage, Hansika; Bergmeir, Christoph; Bandara, Kasun (2020). "Recurrent Neural Networks for Time Series Forecasting: Current Status and Future Directions". *International Journal of Forecasting*. **37**: 388–427. arXiv:1909.00590 (https://arxiv.org/abs/1909.00590). doi:10.1016/j.ijforecast.2020.06.008 (https://doi.org/10.1016%2Fj.ijforecast.2020.06.008). S2CID 202540863 (https://api.semanticscholar.org/CorpusID:202540863).
118. Graves, Alex; Schmidhuber, Jürgen (2005). "Framewise phoneme classification with bidirectional LSTM and other neural network architectures". *Neural Networks*. **18** (5–6): 602–610. doi:10.1016/j.neunet.2005.06.042 (https://doi.org/10.1016%2Fj.neunet.2005.06.042). PMID 16112549 (https://pubmed.ncbi.nlm.nih.gov/16112549). S2CID 1856462 (https://api.semanticscholar.org/CorpusID:1856462).
119. Fernández, Santiago; Graves, Alex; Schmidhuber, Jürgen (2007). "An Application of Recurrent Neural Networks to Discriminative Keyword Spotting" (http://dl.acm.org/citation.cfm?id=1778066.1778092). *Proceedings of the 17th International Conference on Artificial Neural Networks*. ICANN'07. Berlin, Heidelberg: Springer-Verlag. pp. 220–229. ISBN 978-3-540-74693-5.
120. Graves, Alex; Mohamed, Abdel-rahman; Hinton, Geoffrey E. (2013). "Speech recognition with deep recurrent neural networks". *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*. pp. 6645–9. arXiv:1303.5778 (https://arxiv.org/abs/1303.5778). doi:10.1109/ICASSP.2013.6638947 (https://doi.org/10.1109%2FICASSP.2013.6638947). ISBN 978-1-4799-0356-6. S2CID 206741496 (https://api.semanticscholar.org/CorpusID:206741496).
121. Chang, Edward F.; Chartier, Josh; Anumanchipalli, Gopala K. (24 April 2019). "Speech synthesis from neural decoding of spoken sentences" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9714519). *Nature*. **568** (7753): 493–8. Bibcode:2019Natur.568..493A (https://ui.adsabs.harvard.edu/abs/2019Natur.568..493A). doi:10.1038/s41586-019-1119-1 (https://doi.org/10.1038%2Fs41586-019-1119-1). ISSN 1476-4687 (https://search.worldcat.org/issn/1476-4687). PMC 9714519 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9714519). PMID 31019317 (https://pubmed.ncbi.nlm.nih.gov/31019317). S2CID 129946122 (https://api.semanticscholar.org/CorpusID:129946122).

122. Moses, David A.; Metzger, Sean L.; Liu, Jessie R.; Anumanchipalli, Gopala K.; Makin, Joseph G.; Sun, Pengfei F.; Chartier, Josh; Dougherty, Maximilian E.; Liu, Patricia M.; Abrams, Gary M.; Tu-Chan, Adelyn; Ganguly, Karunesh; Chang, Edward F. (2021-07-15). "Neuroprosthesis for Decoding Speech in a Paralyzed Person with Anarthria" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8972947). *New England Journal of Medicine*. **385** (3): 217–227. doi:10.1056/NEJMoa2027540 (https://doi.org/10.1056%2FNEJMoa2027540). PMC 8972947 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8972947). PMID 34260835 (https://pubmed.ncbi.nlm.nih.gov/34260835).
123. Malhotra, Pankaj; Vig, Lovekesh; Shroff, Gautam; Agarwal, Puneet (April 2015). "Long Short Term Memory Networks for Anomaly Detection in Time Series" (https://books.google.com/books?id=USGLCgAAQBAJ&pg=PA89). *European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning – ESANN 2015*. Ciaco. pp. 89–94. ISBN 978-2-87587-015-5.
124. "Papers with Code - DeepHS-HDRVideo: Deep High Speed High Dynamic Range Video Reconstruction" (https://paperswithcode.com/paper/deephs-hdrvideo-deep-high-speed-high-dynamic). *paperswithcode.com*. Retrieved 2022-10-13.
125. Maity, Abhishek; Tukarul, Viraj (23 January 2026). "Forecasting Energy Consumption using Recurrent Neural Networks: A Comparative Analysis". arXiv:2601.17110 (https://arxiv.org/abs/2601.17110) [cs.CY (https://arxiv.org/archive/cs.CY)].
126. Gers, Felix A.; Schraudolph, Nicol N.; Schmidhuber, Jürgen (2002). "Learning precise timing with LSTM recurrent networks" (http://www.jmlr.org/papers/volume3/gers02a/gers02a.pdf) (PDF). *Journal of Machine Learning Research*. **3**: 115–143.
127. Eck, Douglas; Schmidhuber, Jürgen (2002-08-28). "Learning the Long-Term Structure of the Blues". *Artificial Neural Networks — ICANN 2002*. Lecture Notes in Computer Science. Vol. 2415. Berlin, Heidelberg: Springer. pp. 284–289. doi:10.1007/3-540-46084-5_47 (https://doi.org/10.1007%2F3-540-46084-5_47). ISBN 978-3-540-46084-8.
128. Schmidhuber, Jürgen; Gers, Felix A.; Eck, Douglas (2002). "Learning nonregular languages: A comparison of simple recurrent networks and LSTM". *Neural Computation*. **14** (9): 2039–2041. doi:10.1162/089976602320263980 (https://doi.org/10.1162%2F089976602320263980). PMID 12184841 (https://pubmed.ncbi.nlm.nih.gov/12184841). S2CID 30459046 (https://api.semanticscholar.org/CorpusID:30459046).
129. Pérez-Ortiz, Juan Antonio; Gers, Felix A.; Eck, Douglas; Schmidhuber, Jürgen (2003). "Kalman filters improve LSTM network performance in problems unsolvable by traditional recurrent nets". *Neural Networks*. **16** (2): 241–250. doi:10.1016/s0893-6080(02)00219-8 (https://doi.org/10.1016%2Fs0893-6080%2802%2900219-8). PMID 12628609 (https://pubmed.ncbi.nlm.nih.gov/12628609).
130. Graves, Alex; Schmidhuber, Jürgen (2009). "Offline Handwriting Recognition with Multidimensional Recurrent Neural Networks" (http://papers.neurips.cc/paper/3449-offline-handwriting-recognition-with-multidimensional-recurrent-neural-networks.pdf) (PDF). *Advances in Neural Information Processing Systems*. Vol. 22, NIPS'22. MIT Press. pp. 545–552.
131. Graves, Alex; Fernández, Santiago; Liwicki, Marcus; Bunke, Horst; Schmidhuber, Jürgen (2007). "Unconstrained Online Handwriting Recognition with Recurrent Neural Networks" (http://dl.acm.org/citation.cfm?id=2981562.2981635). *Proceedings of the 20th International Conference on Neural Information Processing Systems*. Curran Associates. pp. 577–584. ISBN 978-1-60560-352-0.
132. Baccouche, Moez; Mamalet, Franck; Wolf, Christian; Garcia, Christophe; Baskurt, Atilla (2011). "Sequential Deep Learning for Human Action Recognition". In Salah, Albert Ali; Lepri, Bruno (eds.). *Human Behavior Unterstanding*. Lecture Notes in Computer Science. Vol. 7065. Amsterdam, Netherlands: Springer. pp. 29–39. doi:10.1007/978-3-642-25446-8_4 (https://doi.org/10.1007%2F978-3-642-25446-8_4). ISBN 978-3-642-25445-1.

133. Hochreiter, Sepp; Heusel, Martin; Obermayer, Klaus (2007). "Fast model-based protein homology detection without alignment" (https://doi.org/10.1093%2Fbioinformatics%2Fbtm247). *Bioinformatics*. **23** (14): 1728–1736. doi:10.1093/bioinformatics/btm247 (https://doi.org/10.1093%2Fbioinformatics%2Fbtm247). PMID 17488755 (https://pubmed.ncbi.nlm.nih.gov/17488755).
134. Thireou, Trias; Reczko, Martin (July 2007). "Bidirectional Long Short-Term Memory Networks for Predicting the Subcellular Localization of Eukaryotic Proteins". *IEEE/ACM Transactions on Computational Biology and Bioinformatics*. **4** (3): 441–446. Bibcode:2007ITCBB...4..441T (https://ui.adsabs.harvard.edu/abs/2007ITCBB...4..441T). doi:10.1109/tcbb.2007.1015 (https://doi.org/10.1109%2Ftcbb.2007.1015). PMID 17666763 (https://pubmed.ncbi.nlm.nih.gov/17666763). S2CID 11787259 (https://api.semanticscholar.org/CorpusID:11787259).
135. Tax, Niek; Verenich, Ilya; La Rosa, Marcello; Dumas, Marlon (2017). "Predictive Business Process Monitoring with LSTM Neural Networks". *Advanced Information Systems Engineering*. Lecture Notes in Computer Science. Vol. 10253. pp. 477–492. arXiv:1612.02130 (https://arxiv.org/abs/1612.02130). doi:10.1007/978-3-319-59536-8_30 (https://doi.org/10.1007%2F978-3-319-59536-8_30). ISBN 978-3-319-59535-1. S2CID 2192354 (https://api.semanticscholar.org/CorpusID:2192354).
136. Choi, Edward; Bahadori, Mohammad Taha; Schuetz, Andy; Stewart, Walter F.; Sun, Jimeng (2016). "Doctor AI: Predicting Clinical Events via Recurrent Neural Networks" (http://proceedings.mlr.press/v56/Choi16.html). *JMLR Workshop and Conference Proceedings*. **56**: 301–318. arXiv:1511.05942 (https://arxiv.org/abs/1511.05942). PMC 5341604 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5341604). PMID 28286600 (https://pubmed.ncbi.nlm.nih.gov/28286600).
137. "Artificial intelligence helps accelerate progress toward efficient fusion reactions" (https://www.princeton.edu/news/2017/12/15/artificial-intelligence-helps-accelerate-progress-toward-efficient-fusion-reactions). *Princeton University*. Retrieved 2023-06-12.

Further reading

- Mandic, Danilo P.; Chambers, Jonathon A. (2001). *Recurrent Neural Networks for Prediction: Learning Algorithms, Architectures and Stability*. Wiley. ISBN 978-0-471-49517-8.
- Grossberg, Stephen (2013-02-22). "Recurrent Neural Networks" (https://doi.org/10.4249%2Fscholarpedia.1888). *Scholarpedia*. **8** (2): 1888. Bibcode:2013SchpJ...8.1888G (https://ui.adsabs.harvard.edu/abs/2013SchpJ...8.1888G). doi:10.4249/scholarpedia.1888 (https://doi.org/10.4249%2Fscholarpedia.1888). ISSN 1941-6016 (https://search.worldcat.org/issn/1941-6016).
- Recurrent Neural Networks (http://www.idsia.ch/~juergen/rnn.html). List of RNN papers by Jürgen Schmidhuber's group at Dalle Molle Institute for Artificial Intelligence Research.

Retrieved from "https://en.wikipedia.org/w/index.php?title=Recurrent_neural_network&oldid=1369819153"